

Ablation Study TODO

This document tracks the tasks required to run full ablations comparing Tracks A / B / C on validation (and test) splits, producing the tables and figures for the thesis. All commands reference scripts under `local_extraction/trackA` , `local_extraction/trackB` , and `local_extraction/trackC` .

1. Preparation

1.1 Track A (Two-Stage Baseline)

- ☐ Frames exist under `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg` .
- ☐ GT labels (if oracle mode) under `local_extraction/v2/yolo_labels_540/{clips|videos}/<uid>/<frame:07d>.txt` .
- ☐ Head manifests under `local_extraction/v2/manifests/head_*_{clip|video}.json[1]` (optional for semantic labels).
- ☐ Stage A script configured: `local_extraction/trackA/trackA_stageA/trackA_stageA.py` (edit `RUN_NAME` , `DETECTION_MODE` , `K` , `LAST_FRAME_ONLY` , etc.).
- ☐ Stage B script configured: `local_extraction/trackA/trackA_stageB/trackA_stageB.py` (edit `CANDIDATES_JSONL` , `WRITE_HEAD_TRAIN_VAL` , etc.).

1.2 Track B (Fusion Head)

- ☐ Stage B manifests ready (`head_train.jsonl` , `head_val.jsonl`) from Track A Stage B run or `local_extraction/v2/manifests/` .
- ☐ Train config set in `local_extraction/trackB/trackB_train_loader.py` (`TrainConfig`).
- ☐ Eval config set in `local_extraction/trackB/trackB_eval.py` (`EvalConfig`).
- ☐ Checkpoint expected at `local_extraction/runs/Track_B/checkpoints/trackB_best.pt` .

1.3 Track C (RGTP Pruning)

- ☐ Track B checkpoint available.
 - ☐ Val manifest resolvable (auto-discovery or set `RuntimeConfig.val_manifest`).
 - ☐ Pruning toggles ready: `pruning_enabled` , `rgtp_rate` , `min_keep` in `local_extraction/trackC/trackC_pruning.py` .
 - ☐ Instrumentation toggles (`measure_latency` , `measure_vram` , `measure_flops`) if runtime metrics needed.
-

2. Evaluation Runs

2.1 Track A — Stage A (Detector Proposals)

Edit toggles in `trackA_stageA.py` , then run:

```
python local_extraction/trackA/trackA_stageA/trackA_stageA.py
```

Outputs under `local_extraction/runs/Track_A/<RUN_NAME>/` :

- `candidates.jsonl` , `summary.json` , optional per-image CSVs.

2.2 Track A — Stage B (Crops & Head Manifests)

Edit toggles in `trackA_stageB.py` , then run:

```
python local_extraction/trackA/trackA_stageB/trackA_stageB.py
```

Outputs under `local_extraction/runs/Track_A/trackA_stageB_<ts>/` :

- `crops/` , `manifest.jsonl` , `head_train.jsonl` , `head_val.jsonl` , recall stats.

2.3 Track A — Recall@K Sweep (Optional)

Run the K-sweep helper to measure detector recall:

```
python local_extraction/trackA/trackA_stageA/sweep_k_metrics.py
```

2.4 Track B — Train

Edit `TrainConfig` in `trackB_train_loader.py` (epochs, batch, lr, multi-task toggles, etc.), then run:

```
python local_extraction/trackB/trackB_train_loader.py
```

Checkpoints saved to `local_extraction/runs/Track_B/checkpoints/` .

2.5 Track B — Eval (Val)

Edit `EvalConfig` in `trackB_eval.py` , then run:

```
python local_extraction/trackB/trackB_eval.py
```

Outputs under `local_extraction/runs/Track_B/` :

- `metrics/metrics_val_<ts>.json` , `predictions/` , `overlays/val/` .

2.6 Track C — Pruning Eval (Multiple Rates)

For each pruning rate, edit `RuntimeConfig.rgtp_rate` in `trackC_pruning.py` :

Rate	Setting
0.0 (baseline)	<code>pruning_enabled=False</code> or <code>rgtp_rate=0</code>
0.4	<code>rgtp_rate=0.4</code>
0.5	<code>rgtp_rate=0.5</code>
0.6	<code>rgtp_rate=0.6</code>

Then run:

```
python local_extraction/trackC/trackC_pruning.py
```

Outputs under `local_extraction/runs/Track_C/metrics/` :

- `trackC_val_rateXX_<ts>.json` , `trackC_val_rateXX_<ts>_summary.json` .

2.7 Track C — Compare Metrics

Compare Track B vs Track C side-by-side:

```
python local_extraction/trackC/trackC_compare_metrics.py
```

2.8 Track C — Plot Metrics Over Time

Aggregate all Track C runs into a figure:

```
python local_extraction/trackC/trackC_plots.py --runs_dir
local_extraction/runs/Track_C --out_dir local_extraction/runs/Track_C/plots
```

Outputs: `trackC_metrics_over_time.png` , `trackC_metrics_summary.tsv` .

3. Metrics to Collect

Metric	Description	Source
--------	-------------	--------

Metric	Description	Source
Recall@K	Candidate recall at $K \in \{6, 8, 10\}$	Stage A <code>summary.json</code> or sweep script
accuracy	Candidate-level accuracy	Track B/C metrics JSON
mAP	Candidate-level mean AP	Track B/C metrics JSON
ttc_mae_seconds	TTC mean absolute error (sec)	Track B/C metrics JSON
N mAP	Frame-level noun correct + $\text{IoU} \geq 0.5$	Track B/C metrics JSON
N+V mAP	Frame-level noun+verb correct	Track B/C metrics JSON
N+ δ mAP	Frame-level noun+TTC-bin correct	Track B/C metrics JSON
All mAP	Frame-level noun+verb+TTC-bin correct	Track B/C metrics JSON
N_top5_acc / mAP	Top-5 hit/AP for noun	Track B/C metrics JSON
latency_ms_mean	Inference time per sample (ms)	Track C metrics JSON (instrumentation)
peak_vram_bytes	Peak GPU memory	Track C metrics JSON (instrumentation)
head_flops	Head FLOPs	Track C metrics JSON (instrumentation)
throughput_samples_per_s	Throughput	Track C metrics JSON (instrumentation)

4. Tables

Table 1: Track Comparison (Val)

Track	accuracy	mAP	N mAP	N+V mAP	N+ δ mAP	All mAP	Latency (ms)
A (Stage B only)	—	—	—	—	—	—	—
B (fusion)							
C @0.5							

Table 2: Pruning Rate Ablation (Val, Track C)

Rate	accuracy	mAP	N mAP	Latency (ms)	Speedup
0.0 (baseline)					1.00×
0.4					
0.5					

Rate	accuracy	mAP	N mAP	Latency (ms)	Speedup
0.6					

Table 3: Recall@K (Stage A Detector)

K	Val Recall
6	
8	
10	

Table 4: Multi-Task Semantic Metrics (Val)

Track	N_top5_acc	Nv_top5_acc	N_delta_top5_acc	All_top5_acc
B				
C @0.5				

5. Figures

- ☐ **Figure 1:** Bar chart comparing mAP / N mAP across Tracks A/B/C (val).
- ☐ **Figure 2:** Line plot of mAP vs pruning rate (Track C ablation).
- ☐ **Figure 3:** Scatter plot of Latency vs N+V mAP (Pareto frontier).
- ☐ **Figure 4:** Attention visualization samples (Track B fusion, Track C pruned tokens).
- ☐ **Figure 5:** Recall@K curve for Stage A detector (sweep).
- ☐ **Figure 6:** `trackC_metrics_over_time.png` from Track C plots script.

6. Aggregation Script (Optional)

Create `scripts/aggregate_ablations.py` to:

- Load all JSON reports from `runs/Track_B/metrics/*.json` and `runs/Track_C/metrics/*.json` .
- Merge into a single DataFrame with track/rate columns.
- Export summary tables as CSV and LaTeX.
- Generate matplotlib figures and save to `runs/ablations/figures/` .

7. Checklist

- ☐ Track A Stage A + Stage B runs completed; manifests generated.
- ☐ Track B training completed; `trackB_best.pt` saved.
- ☐ Track B eval run completed; metrics JSON present.
- ☐ Track C pruning runs completed for rates 0.0, 0.4, 0.5, 0.6.
- ☐ Track C compare and plots scripts run.

- ☐ JSON reports validated (no NaN, consistent keys).
 - ☐ Tables filled and cross-checked.
 - ☐ Figures exported as PDF/PNG for thesis.
 - ☐ Results discussed in thesis chapter (limitations, insights).
-

Notes

- All scripts are configured via in-file dataclasses (no CLI args by default); edit toggles directly.
- Use `measure_latency=True` in Track C for timing; CUDA events preferred if GPU available.
- Set `num_workers=0` in datasets to avoid tokenizer pickling issues.
- Keep descriptive run names and commit hashes for reproducibility.
- Store raw predictions for failure analysis if needed.